

the queue, the kernel first tries to enlarge an existing request by merging the new request with one that is already in the queue. If the request cannot be merged with an existing one, then the new request will be assigned a position in the queue based on several factors including the elevator algorithm (which we will configure later).

First, let's check what the current queue length is and which current IO scheduler the OS is using. To view the current queue length, issue the following command:

```
# cat /sys/block/sda/queue/nr_requests
128
```

You can see that my current queue length is 128 requests. Now I want to increase it to 170—let's do it using the *echo*:

```
# echo 170 > /sys/block/sda/queue/nr_requests
# cat /sys/block/sda/queue/nr_requests
170
```

To make it permanent, add the entry to the */etc/rc.local* file. [As a rule, keep in mind that anything under */sys* can be made permanent by using */etc/rc.local*.]

Now, let's check which elevator (or I/O scheduler) algorithm the OS is using by default:

```
# cat /sys/block/sda/queue/scheduler
noop anticipatory deadline [cfq]
```

On my system, it's *cfq*. Now the question is...

## What is an I/O scheduler?

I/O schedulers control the way Linux handles reads and writes to disks. Linux 2.6 includes a number of I/O schedulers—the intention of providing these choices is to allow better optimisation for different classes of workload.

Without an I/O scheduler, Linux follows FIFO (First In, First Out). This could result in massive HDD thrashing—for example, if one process is reading from one part of the disk and one writing to another, the heads would have to seek back and forth across the disk for every operation. The scheduler's main goal is to optimise disk access times.

This is where I/O schedulers come into the picture. Quoting the WlugWiki [[www.wlug.org.nz/LinuxIOScheduler](http://www.wlug.org.nz/LinuxIOScheduler)] an I/O scheduler can use the following techniques to improve performance:

- Request merging—The scheduler merges adjacent requests together to reduce disk seeking.
- Elevator—The scheduler orders requests based on their physical location on the block device, and it basically tries to seek in one direction as much as possible.
- Prioritisation—The scheduler has complete control over how it prioritises requests, and can do so in a number of ways

In addition to these, all I/O schedulers also take into account resource starvation, which ensures requests to

be serviced.

Scheduler algorithms operate like elevators in buildings, and are therefore also called elevators. As per *The Red Hat Enterprise Linux 5 IO Tuning Guide*: “The algorithms used to operate real-life building elevators make sure that it services requests per floor, efficiently. To be efficient, the elevator does not travel to each floor depending on which one issued a request to go up or down, first. Instead, it moves in one direction at a time, taking as many requests as it can until it reaches the highest or lowest floor, then does the same in the opposite direction. Simply put, these algorithms schedule disk I/O requests according to which logical block address on the disk they are targeted to. This is because the most efficient way to access the disk is to keep the access pattern as sequential (i.e., moving in one direction) as possible. Sequential, in this case, means ‘by increasing the logical block address number’. As such, a disk I/O request targeted for disk block 100 will normally be scheduled before a disk I/O request targeted for disk block 200. This is typically the case, even if the disk I/O request for disk block 200 was issued first.”

There are currently four types of schedulers available:

1. The no-op scheduler (noop)
2. The anticipatory IO scheduler (anticipatory)
3. The deadline scheduler (deadline)
4. The Complete Fair Queueing scheduler (CFQ)

## The no-op scheduler

As the name suggests, ‘no-op’ actually does nothing—it simply works in the FIFO manner. No configurable options are available here. The I/O requests are sent in the same manner as they are received, and it is left to the hardware to work on it. The main goal is to conserve CPU cycles.

You can make no-op your IO scheduler as follows:

```
echo noop > /sys/block/sda/queue/scheduler
```

To make it your choice of I/O scheduler across reboots, edit */boot/grub/grub.conf* and append *elevator=noop* at the end of the kernel line:

```
root (hd0,0)
kernel /vmlinuz-2.6.29-6.217.2.16.fc11.i686.PAE ro root=LABEL=^Fedora rhgb quiet
elevator=noop
```

## The anticipatory scheduler

I personally think this is the most optimistic of the schedulers. It waits in anticipation that the next I/O request might be for the next disk block, before moving to another location. The basic idea is that the IO scheduler will wait for a certain short period of time after catering to one I/O request and before going on to the next I/O request. So if the next I/O request is for the next disk block, it saves time in moving the head back to the same location again. But this may also result in additional latency, if the next I/O is not for the next disk block.

Switch to this scheduler as follows: